yxb 的任务管理器

系统说明文档

项目	简化的任务管理器
项目名称	yxb 的任务管理器（基于 psutil 的操作系统监控程序）
课程/实验	Python 网络编程实验一
作者	闫晓斌
学号	20243915
版本	v1.0.0
开发语言	Python 3.13.13
代码仓库	https://github.com/xiaobin15m/yxb-task-manager

2026 年 11 月

目 录

1. 项目概述.....4
1.1 运行环境.....4
1.2 交付物清单.....4
2. 所用的 AI 工具.....6
2.1 工具与用途.....6
2.2 主要提示词（摘要）.....6
2.3 AI 生成内容的人工复核.....7
3. 开发框架与技术.....8
3.1 为什么自绘图表.....8
3.2 界面样式来源.....8
4. 系统设计.....9
4.1 总体架构.....9
4.2 模块说明.....9
4.3 关键实现.....10
4.3.1 速率计算.....10
4.3.2 进程采样线程（性能取舍）.....10
4.3.3 界面还原.....11
4.3.4 图表绘制.....11
4.4 指标与刷新策略.....12
4.5 界面结构.....12
4.6 界面效果.....13
5. 实现的主要功能.....15

6. 用户操作手册.....16
6.1 安装.....16
6.2 启动.....16
6.3 界面说明.....16
6.3.1 顶部.....16
6.3.2 性能页.....16
6.3.3 进程页.....17
6.3.4 状态栏.....17
6.4 常用操作.....17
6.5 常见问题（FAQ）.....18
7. 测试与验证.....19
7.1 单元测试.....19
7.2 界面与交互验证.....19
7.3 已知限制.....20
8. 代码仓库与提交说明.....21
8.1 仓库地址.....21
8.2 创建仓库与推送.....21
8.3 推送本地代码.....21
8.4 上传后核对.....22
8.5 压缩包说明.....22
8.6 作业要求对照表.....22
附录 A：完整目录结构.....24
附录 B：核心代码片段.....25

1. 项目概述

本项目实现了一个带图形用户界面的系统监控程序，用于实时观测操作系统的运行状态。程序以 psutil 作为底层数据源，以 tkinter / ttk 构建桌面界面，界面按 Windows 任务管理器的组织方式设计：顶部“进程 / 性能”标签、左侧组件列表、右侧大图表与统计列、底部概览状态栏。

监控能力包括：

1. CPU 总体使用率与每个逻辑核心的使用率；
2. 物理内存、交换分区使用率；
3. 网络接收 / 发送速率与累计流量，网卡连接状态；
4. 磁盘 IO 读取 / 写入速率；
5. 各磁盘分区的容量使用率；
6. 进程列表：名称、状态、CPU 占比、内存占用、PID。

1.1 运行环境

项目	版本 / 说明
操作系统	Windows 11，亦可运行于 Linux / macOS
Python	3.13.13（MSC v.1944 64-bit）
GUI 库	tkinter / Tk 8.6 + ttk（Python 标准库）
采集库	psutil 7.2.2
测试框架	unittest（标准库）
版本管理	Git 2.56 + GitHub

1.2 交付物清单

交付物	位置
全部源代码	main.py、yxbtaskmgr/、tests/
依赖清单	requirements.txt
启动脚本	run.bat

交付物	位置
项目说明	README.md
系统文档（Word 版）	docs/系统说明文档.docx
系统文档（Markdown 版）	docs/系统说明文档.md
界面截图	docs/images/main-window.png、docs/images/processes.png
压缩提交包	yxb 的任务管理器_系统监控_源码与文档.zip

2. 所用的 AI 工具

2.1 工具与用途

AI 工具	使用方式	具体用途
Trae（TraeCode）AI 编程助手（Agent 模式） Deepseek Harness	在对话中描述需求，由 AI 读取 / 创建工程文件并执行命令	需求拆解、工程骨架、采集线程与界面代码生成、界面样式重做、缺陷定位、测试与文档草稿

AI 在本次开发中的典型贡献：

- 生成 SystemCollector 主采样线程，给出“累计计数器差分 + time.monotonic() 时间差”的速率计算方案；
- 按“仿任务管理器”的要求重做界面：顶层“进程 / 性能”标签、左侧组件列表、右侧面积图 + 统计列、底部状态栏、ttk 进程表；
- 用自绘 Canvas 实现面积图、迷你趋势图、核心网格、使用率条、网卡行等控件，避免引入 matplotlib；
- 定位并修复“只有一个采样点时 create_line 坐标不足 4 个”导致的 TclError；
- 实测 psutil.process_iter 读取名称 / 状态约需 1.7 秒，改为独立进程采样线程 + 名称缓存 + 仅对展示条目取状态，避免拖慢 1 秒级主采样；
- 修复 memory 视图 grid_columnconfigure(weight=...) 使用浮点数导致的 TclError，并处理统计列内容被裁切、底部列表与标题之间出现空白等布局问题；
- 生成单元测试用例并执行验证，起草 README 与本文档，交由人工核对。

2.2 主要提示词（摘要）

1. “基于 psutil 模块实现系统监控程序，监控磁盘使用率、CPU 占用率、网络收发速率、磁盘 IO 读写速率，要求有 GUI，代码提交到 GitHub，并完成系统文档。”
2. “采集放到后台线程，界面不要卡住；窗口在不同分辨率下不要被裁切。”

3. “补充单元测试和系统说明文档，文档要包含 AI 工具、技术框架、功能说明和用户操作手册。”

2.3 AI 生成内容的人工复核

- **静态检查**：逐文件阅读代码，检查导入、类型标注、异常分支与线程退出逻辑；
- **语法编译**：执行 `python -m compileall yxbtaskmgr main.py tests`；
- **单元测试**：执行 `python -m unittest discover -s tests -v`，8 个用例全部通过；
- **几何核对**：用临时脚本打印控件尺寸，确认统计列最后一行的数值未被裁切；
- **实际运行截图**：启动 GUI 等待 10 秒积累采样点，逐页截图检查图表、列表、状态栏数值；
- **性能实测**：对 `psutil.process_iter` 的不同属性组合计时，确定进程采样策略；
- **语义核对**：对照 psutil 文档确认 `cpu_percent(interval=None)`、`nowrap=True`、`Process.cpu_percent()` 的语义。

3. 开发框架与技术

技术	作用
psutil	跨平台读取 CPU、内存、磁盘、网络、分区、进程等系统指标
tkinter / ttk	构建主窗口、标签栏、下拉框、按钮与进程表格；图表、进度条使用 Canvas 自绘
threading + queue	主采样线程、进程采样线程与 UI 主线程解耦，队列只保留最新快照
dataclasses	以不可变数据类 Snapshot / PartitionUsage / NetAdapter / ProcessEntry 传递数据
ctypes（仅 Windows）	启用 DPI 感知，避免高缩放比例下界面模糊
unittest	格式化函数与采集线程的单元测试
Git + GitHub	源代码版本管理与远程托管

3.1 为什么自绘图表

- tkinter 属于 Python 标准库，安装成本低，符合“只需 pip install psutil 即可运行”的目标；
- matplotlib 嵌入 tkinter 会增加数十 MB 依赖，并带来字体、DPI 兼容问题；
- 需求是固定时间窗口的滚动曲线，Canvas 自绘约 200 行即可实现，启动更快，外观也能对齐任务管理器的细网格与淡色填充。

3.2 界面样式来源

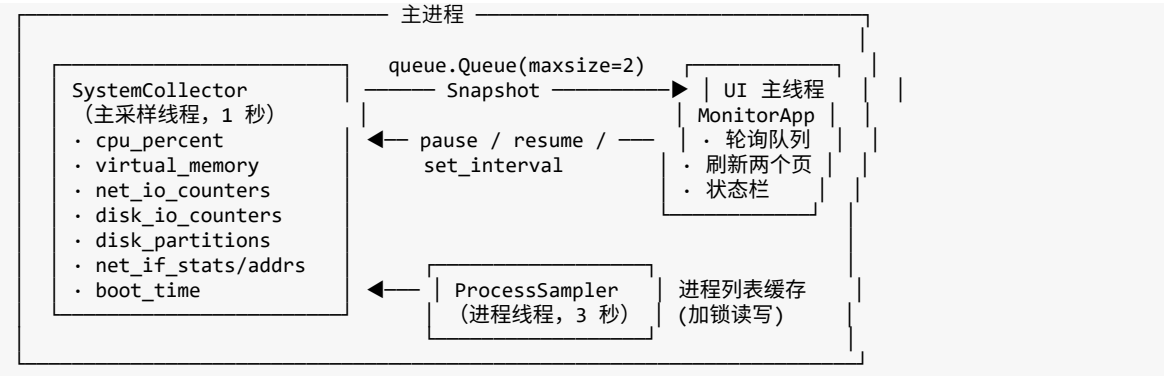
没有使用第三方主题库，而是：

- ttk.Style().theme_use("clam") + 自定义 TM.Treeview / TM.Treeview.Heading / TM.Vertical.TScrollbar / TM.TCombobox 样式，去掉立体边框、改为白底扁平表头；
- 自定义浅色配色：窗口底色 #f3f3f3、内容白底、分隔线 #e2e2e2、强调蓝 #0067c0、图表蓝 #0078d4 与橙 #c2690a；

- 顶部标签用 `Label` + 2 px 下划线实现，选中项蓝色下划线 + 加粗，与任务管理器的标签栏一致。

4. 系统设计

4.1 总体架构



- **主采样线程**：每 interval 秒采集 CPU / 内存 / 网络 / 磁盘 / 分区 / 网卡，结果写入容量为 2 的队列，队列满时丢弃旧数据；
- **进程采样线程**：每 3 秒遍历一次进程，结果保存在线程内的加锁缓存中，主采样线程生成快照时直接读取缓存；
- **界面主线程**：每 250 ms 轮询队列，只做界面更新，不执行任何阻塞式 psutil 调用；
- 磁盘分区、网络适配器变化缓慢，默认 10 秒刷新一次。

4.2 模块说明

文件	职责
main.py	程序入口，创建 MonitorApp 并进入 mainloop
yxbtaskmgr/collector.py	数据类定义；SystemCollector 主采样线程；ProcessSampler 进程采样线程；速率计算；分区 / 网卡读取容错
yxbtaskmgr/format.py	字节容量、速率、图表刻度、运行时长、进程状态中文化的格式化
yxbtaskmgr/ui/app.py	窗口与标签栏、ttk 样式、页面切换、状态栏、暂停/间隔控制、DPI 适配
yxbtaskmgr/ui/theme.py	任务管理器浅色配色、字体探测、使用率颜色分档

文件	职责
yxbtaskmgr/ui/widgets.py	AreaChart、Sparkline、CoreGrid、StatPanel、NavItem、MeterBar、BarRow、AdapterRow、ProcessTable
yxbtaskmgr/ui/performance.py	“性能”页：组件列表、四组图表与统计、核心网格、内存组成、分区与网卡列表
yxbtaskmgr/ui/processes.py	“进程”页：表格、表头排序、名称筛选
tests/test_yxbtaskmgr.py	格式化与采集线程的单元测试

4.3 关键实现

4.3.1 速率计算

网络与磁盘 IO 使用累计字节计数器，速率按两次采样的差值计算：

```
rate = (counter_now - counter_last) / (t_now - t_last)
```

- 使用 psutil.net_io_counters(nowrap=True)、disk_io_counters(nowrap=True)，计数器溢出时自动修正；
- 差值小于 0 时按 0 处理；
- 采集线程启动时建立基线，暂停期间持续重置基线，恢复后不会出现速率尖峰；
- 启动前调用一次 cpu_percent(interval=None) 预热，避免第一帧恒为 0。

4.3.2 进程采样线程（性能取舍）

在 Windows 上实测 psutil.process_iter(["pid","name","status","cpu_percent","memory_info"]) 遍历约 240 个进程需 **1.7 秒**（其中名称约 0.66 秒、状态约 0.5 秒）。如果把这段逻辑放进 1 秒级的主采样线程，会导致采样间隔被拉长、曲线出现断档。

处理方式：

- 进程扫描放到独立守护线程 ProcessSampler，每 3 秒一次，与主采样节奏互不影响；

- 扫描时只请求 pid、cpu_percent、memory_info 三个属性；
- 进程名按 PID 缓存（同一 PID 的名称不会变化），退出进程的缓存会被清理；
- 只对排序后将要展示的前 80 个进程调用 Process.status()；
- 结果放在 threading.Lock 保护的缓存里，主采样线程读取后写入 Snapshot。

实测优化后单次扫描约 1.0 秒，且发生在独立线程，主采样仍保持 1 秒间隔。

进程 CPU 数值按逻辑核心数归一化（cpu_percent / cpu_count），与任务管理器“占全部 CPU 的百分比”一致；否则多核进程会显示 300%、1300% 之类的值。

4.3.3 界面还原

任务管理器元素	本项目实现
顶部标签栏	Label + 2 px 下划线，选中项加粗、蓝色下划线
左侧组件列表	NavItem：标题、实时值、迷你趋势图，选中项浅蓝底 + 3 px 蓝条
右侧大图表	AreaChart：60 秒窗口、5 条横向网格、6 条纵向网格、左下“60 秒”、单条曲线淡色面积填充
右侧统计列	StatPanel：标签在上（灰）、数值在下（加粗），支持颜色方块区分双曲线
底部概览状态栏	进程 / CPU / 内存 / 磁盘 / 网络，右侧显示采集状态、采样次数、运行时长、版本
进程列表	ttk.Treeview + 自定义表头样式，列宽、行高、锚点对齐任务管理器

其他细节：

- 页面切换用 grid()/grid_remove()，隐藏页面中的图表在 redraw() 开头判断 winfo_ismapped() 直接返回，切回时由 <Map> 事件与显式调用补画，避免不可见控件反复重绘；

- 内存组成条用两个按百分比设置 `grid_columnconfigure(weight=...)` 的 `Frame` 实现，随窗口宽度自适应；
- 窗口尺寸按屏幕分辨率计算，最小 1000×620；Windows 下启用 DPI 感知。

4.3.4 图表绘制

- 采样点保存在 `deque(maxlen=60)` 中，实现固定长度滚动窗口；
- 数据右对齐，最新点在最右侧，横轴步长由“绘图宽度 / 最大点数”决定；
- 纵轴支持固定上限（CPU / 内存 0~100%）与自动取整上限（速率图按 $1/1.5/2/2.5/3/4/5/6/8\times10^{n}$ 取整）；
- 第一条曲线用淡色面积填充，第二条曲线只画折线，保证双曲线互相不遮挡；
- 只有一个采样点时绘制圆点，两个及以上时绘制折线与面积，避免首帧坐标不足报错。

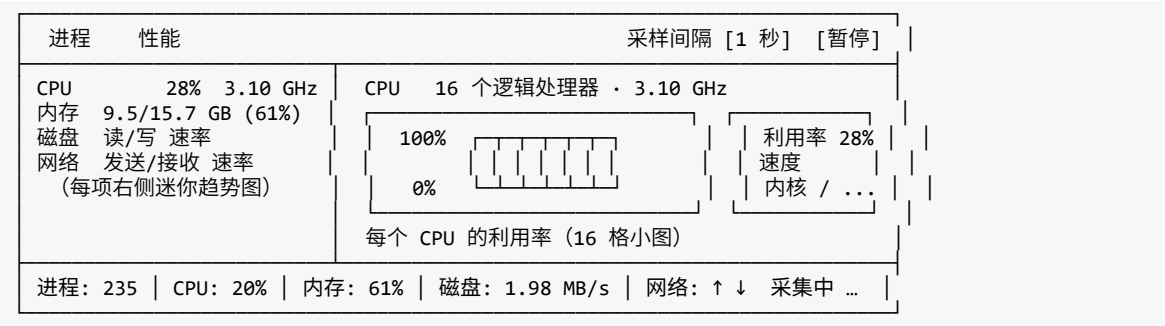
4.4 指标与刷新策略

指标	psutil 接口	刷新频率	展示方式
CPU 总体占用	`cpu_percent(interval=None)`	每个采样周期	大图表 + 统计列 + 左侧列表迷你曲线
每核心占用	`cpu_percent(percpu=True)`	每个采样周期	“每个 CPU 的利用率”核心网格
CPU 主频 / 核心数	`cpu_freq()`、`cpu_count()`	每个采样周期	标题副标题 + 统计列
系统负载 / 运行时长	`getloadavg()`、`boot_time()`	每个采样周期	统计列
内存 / 交换分区	`virtual_memory()`、`swap_memory()`	每个采样周期	大图表 + 内存组成条
网络收发速率	`net_io_counters(nowrap=True)`	每个采样周期	大图表（接收/发送）+ 统计列
网卡状态 / IPv4 / 链路速率	`net_if_stats()`、`net_if_addrs()`	每 10 秒	网络适配器列表
磁盘读写速率	`disk_io_counters(nowrap=True)`	每个采样周期	大图表（读取/写入）+ 统计列

指标	psutil 接口	刷新频率	展示方式
磁盘分区使用率	disk_partitions()、disk_usage()	每 10 秒	分区使用率条 + 百分比 + 容量
进程列表	process_iter()、Process.status()	每 3 秒	进程页表格 + 状态栏进程数

使用率颜色规则：< 75% 蓝色，75%~90% 橙色，≥ 90% 红色（用于使用率条与百分比提示）。

4.5 界面结构



“进程”页把上半部分换成搜索框 + 五列表格（名称 / 状态 / CPU / 内存 / PID），底部同样是概览状态栏。

4.6 界面效果

以下为程序在 Windows 11（Python 3.13.13）下的实际运行截图。

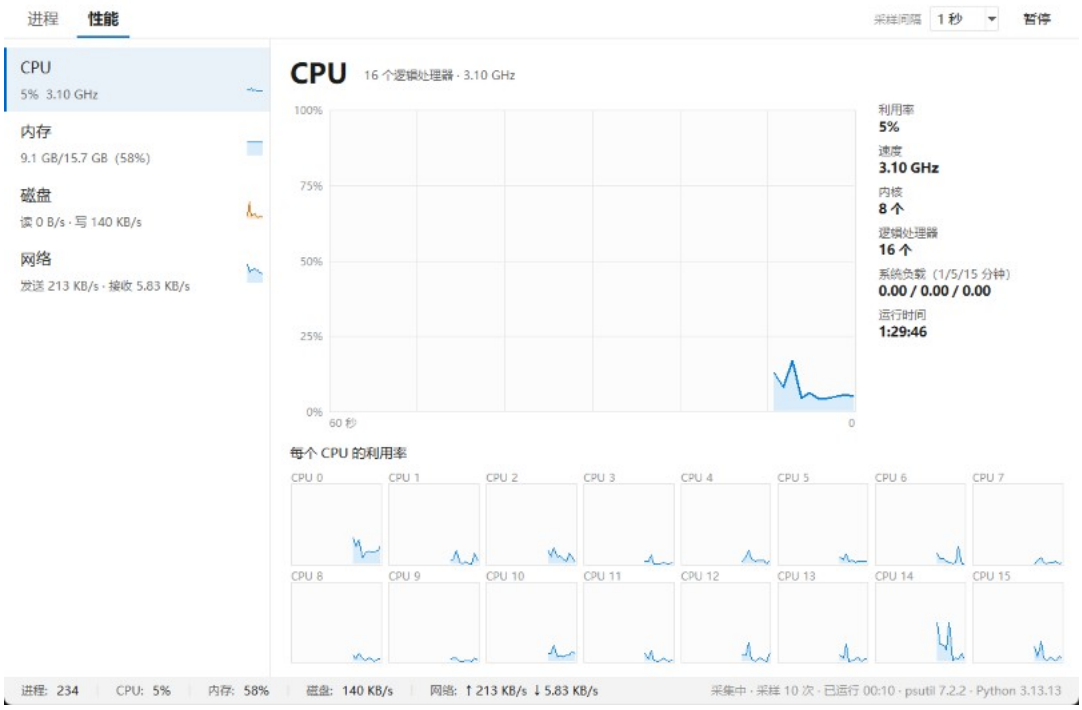


图 1　“性能”页：左侧组件列表、60 秒滚动图表、右侧统计列与每核心利用率

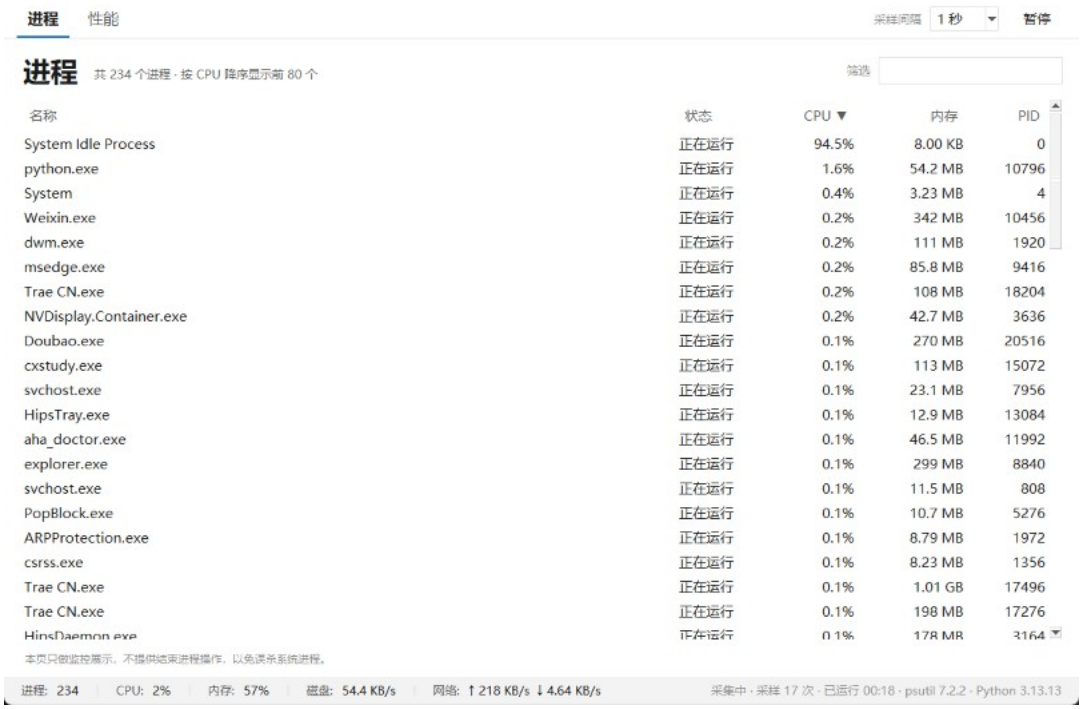


图 2　“进程”页：五列表格、表头排序与名称实时筛选

5. 实现的主要功能

1. **CPU 监控：**总体占用率、每个逻辑核心占用率、主频、物理 / 逻辑核心数、系统负载、运行时长、60 点历史曲线。
2. **内存监控：**物理内存占用率与已用 / 可用 / 总容量、交换分区使用率与容量、内存组成条、历史曲线。
3. **网络监控：**接收 / 发送实时速率（B/s ~ GB/s 自动换算单位）、双曲线、累计流量、网卡连接状态与链路速率。
4. **磁盘 IO 监控：**读取 / 写入实时速率、双曲线、分区数量与总容量。
5. **磁盘分区监控：**盘符、文件系统、使用率条、已用 / 总容量。
6. **进程监控：**名称、中文状态、CPU 占比、内存占用、PID，表头排序与名称筛选。
7. **交互控制：**采样间隔 0.5/1/2/5 秒切换；暂停 / 继续采集；窗口缩放；关闭时安全停止后台线程。
8. **概览状态栏：**进程数、CPU、内存、磁盘速率、网络速率、采集状态、采样次数、运行时长、psutil / Python 版本。

6. 用户操作手册

6.1 安装

1. 安装 Python 3.9 或更高版本，并勾选“Add Python to PATH”；

2. 打开命令行，进入项目目录，执行：

```
pip install -r requirements.txt
```

6.2 启动

```
python main.py
```

Windows 用户也可双击 run.bat。

启动后约 1 秒出现主窗口，默认停留在“性能”页；进程列表因需先建立 CPU 采样基线，约 3 秒后出现 CPU 数值。

6.3 界面说明

6.3.1 顶部

区域	说明
“进程”“性能”标签	点击切换页面，选中项蓝色下划线
采样间隔	下拉选择 0.5 / 1 / 2 / 5 秒，立即生效
暂停	暂停后按钮变“继续”并变为橙色，采集线程停止采样

6.3.2 性能页

区域	说明
左侧组件列表	CPU、内存、磁盘、网络的实时值与迷你曲线；点击切换右侧详情
标题与副标题	当前组件名称与关键信息（核心数、内存总量、分区数、网卡数）

区域	说明
大图表	60 秒滚动窗口；CPU / 内存为百分比，磁盘 / 网络为速率并自动换算单位
统计列	4~6 项指标，双曲线项目带颜色方块
底部详情	CPU：每核心利用率小图；内存：组成条 + 图例；磁盘：分区列表；网络：适配器列表

6.3.3 进程页

区域	说明
进程计数	显示系统进程总数与当前展示条数（默认按 CPU 降序取前 80 个）
筛选框	输入名称片段即时过滤，大小写不敏感
表头	点击任意列排序，再次点击切换升 / 降序，箭头指示当前排序
列表	名称、状态（正在运行 / 已挂起 / 磁盘等待 …）、CPU、内存、PID

6.3.4 状态栏

左侧依次显示进程数、CPU、内存、磁盘速率（读取 + 写入合计）、网络速率（↑ 发送、↓ 接收）；右侧显示“采集中 / 已暂停”、采样次数、运行时长与 psutil / Python 版本。

6.4 常用操作

操作	方法	效果
调整采样频率	点击右上角“采样间隔”下拉框	立即按新间隔采样；曲线窗口仍保留 60 个点
暂停采集	点击右上角“暂停”	按钮变为“继续”，状态栏显示“已暂停”，图表停止滚动
恢复采集	点击“继续”	恢复采样，暂停期间的速率不会出现尖峰

操作	方法	效果
查看不同组件	点击左侧 CPU / 内存 / 磁盘 / 网络	右侧切换为对应大图表与统计信息
切换页面	点击顶部“进程”或“性能”	进程页显示最新一次进程采样结果
筛选进程	在进程页右上角输入框键入名称	即时过滤，计数同步更新
排序进程	点击进程页表头	在 CPU / 内存 / PID / 名称 / 状态之间切换排序
缩放窗口	拖动窗口边缘	图表、核心网格、列表自适应重排，最小 1000×620
关闭程序	点击窗口关闭按钮	先停止两个采集线程再退出，不残留后台进程

6.5 常见问题（FAQ）

问题	原因与解决
`ModuleNotFoundError: No module named 'psutil'`	未安装依赖，执行 `pip install psutil` 或 `pip install -r requirements.txt`
`ModuleNotFoundError: No module named 'tkinter'`	Linux 缺少 Tk，执行 `sudo apt install python3-tk`；Windows 请使用官方 Python 安装包
启动后进程页短暂显示“正在读取进程列表…”	进程采样线程首次扫描需遍历全部进程，约 1 秒后出现数据
进程 CPU 一列最初为 0.0%	`Process.cpu_percent()` 首次调用只建立基线，第二个采样周期后才有数值
磁盘分区列表为空或缺少某个分区	光驱空介质、无权限挂载点、网络盘等会被跳过，属正常现象
网络 / 磁盘速率为 0	当前无网络或磁盘读写活动；可打开浏览器或复制大文件观察曲线变化
首次启动 CPU 显示 0.0%	`cpu_percent(interval=None)` 首次调用没有参照点，程序已预热；若仍出现，等待下一个采样周期即可
系统负载显示 0.00	Windows 上由 psutil 模拟计算，需要程序运行一段时间后才趋于准确

问题	原因与解决
界面字体偏小或模糊	程序已启用 DPI 感知；若系统缩放比例极大，可调整 Windows 缩放设置

7. 测试与验证

7.1 单元测试

执行命令：

```
python -m unittest discover -s tests -v
```

实际结果（本机 Windows 11 / Python 3.13.13）：

用例	内容	结果
test_bytes_units	字节容量格式化（B/KB/GB）	通过
test_rate_suffix	速率单位（B/s、MB/s）	通过
test_axis_format	图表纵轴紧凑格式	通过
test_duration	运行时长格式化（MM:SS、HH:MM:SS）	通过
test_uptime	开机时长格式化（天:时:分:秒）	通过
test_process_status	进程状态中文映射与未知状态回退	通过
test_snapshot_values	采集线程输出合法快照（CPU/内存/网络/磁盘/分区/适配器/进程）	通过
test_pause_and_resume	暂停 / 恢复状态与两个线程的退出	通过

汇总：Ran 8 tests ... OK。

7.2 界面与交互验证

- 启动 GUI 等待 10 秒积累采样点，逐页截图核对：CPU 大图表与核心网格、内存组成条、磁盘分区列表、网络适配器列表均正常显示，状态栏数值与页面数值一致；

- 用临时脚本打印 StatPanel 的子控件位置，确认 6 行统计信息在最小窗口高度下也未被裁切；
- 用临时脚本核对底部区域行权重，修复“磁盘分区”标题与列表之间的大段空白、以及内存组成条位置；
- 在磁盘写入压力下观察“磁盘”图表与左侧列表数值同步变化；在网络下载时观察接收速率上升；
- 进程页验证：CPU 列出现非零值，点击表头可切换排序，输入框筛选后计数同步变化；
- 运行过程中标准错误输出为空，未出现 Tk 异常。

7.3 已知限制

- 网络速率统计的是所有网卡的合计值，未按适配器拆分；
- 磁盘 IO 为物理磁盘合计值，无法按分区区分（受 psutil 接口限制）；
- 进程列表按 CPU 降序只展示前 80 个进程，进程总数单独统计；
- 进程状态为采样瞬间的快照，且只对展示的 80 个进程读取；
- Windows 的系统负载为 psutil 模拟值，精度低于 Linux；
- 历史曲线固定保留最近 60 个采样点，切换间隔后曲线覆盖的时间跨度会变化；
- 进程页只做监控展示，不提供结束进程操作，以免误杀系统进程；
- 程序未打包为 exe，运行需本机安装 Python 与 psutil。

8. 代码仓库与提交说明

8.1 仓库地址

项目	内容
代码托管平台	GitHub
仓库地址	https://github.com/xiaobin15m/yxb-task-manager
可见性	Public
默认分支	main

8.2 创建仓库与推送

1. 登录 GitHub，点击右上角 + → **New repository**；
2. **Repository name** 填 yxb-task-manager；
3. 可见性选择 **Public**；
4. 不勾选 “Add a README file”“Add .gitignore”“Choose a license”——本地已有完整工程，勾选后首次推送会因历史不一致而失败；
5. 点击 **Create repository**，在随后的 “Quick setup” 页面复制仓库地址。

8.3 推送本地代码

本项目目录已经是一个 Git 仓库并有提交记录，不需要重新执行 git init（该命令只用于首次创建仓库）。在项目根目录依次执行：

```
git add .
git commit -m "docs: 系统说明文档改为 GitHub 提交说明"
git branch -M main
git remote add origin https://github.com/xiaobin15m/yxb-task-manager.git
git push -u origin main
```

常见报错处理：

报错	处理方式
remote origin already exists	改用 git remote set-url origin https://github.com/xiaobin15m/yxb-task-manager.git

报错	处理方式
src refspec main does not match any	当前不在 main 分支，先执行 git checkout -b main
Authentication failed / 要求密码	GitHub 已停用账号密码，密码处填 **Personal Access Token**：Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token，勾选 repo 权限后复制
Updates were rejected	远程有本地没有的提交（如初始化时勾选了 README），先 git pull --rebase origin main 再推送
Support for password authentication was removed	同上，改用 token 或 SSH 地址 git@github.com:xiaobin15m/yxb-task-manager.git

8.4 上传后核对

检查项	期望结果
仓库首页	能看到 main.py、yxbtaskmgr/、docs/、tests/、README.md
README 渲染	两张界面截图正常显示，没有裂图
系统文档	点击 docs/系统说明文档.md 可在线预览
提交历史	至少 2~3 条有意义的提交信息
仓库地址	已回填到本文档封面与 README.md

8.5 压缩包说明

yxb 的任务管理器_系统监控_源码与文档.zip 包含全部源代码、requirements.txt、run.bat、README.md、系统文档以及两张界面截图。

8.6 作业要求对照表

要求	完成情况	位置
1. 具备图形用户界面	已实现：tkinter / ttk 桌面 GUI，仿 Windows 任务管理器，含“进程”“性能”两页	python main.py 运行；见第 4.5、6.3 节及 docs/images/ 截图

要求	完成情况	位置
2. 代码提交到 GitHub 或 Gitee	已提交到 GitHub 公开仓库	https://github.com/xiaobin15m/yxb-task-manager
3. 完成系统文档	已包含 AI 工具、开发框架与技术、主要功能、用户操作手册、代码库地址	本文档 docs/系统说明文档.md
4. 源代码与文档压缩提交	已打包	yxb 的任务管理器_系统监控_源码与文档.zip

附录 A：完整目录结构

```
.
├── main.py
├── yxbtaskmgr/
│   ├── __init__.py
│   ├── collector.py
│   ├── format.py
│   └── ui/
│       ├── __init__.py
│       ├── app.py
│       ├── theme.py
│       ├── widgets.py
│       ├── performance.py
│       └── processes.py
├── tests/
│   └── test_yxbtaskmgr.py
├── docs/
│   ├── 系统说明文档.docx
│   ├── 系统说明文档.md
│   └── images/
│       ├── main-window.png
│       └── processes.png
├── requirements.txt
├── run.bat
├── README.md
└── .gitignore
```

附录 B：核心代码片段

速率计算：

```
def _rate(current, previous, elapsed):
    if current is None or previous is None or elapsed <= 0:
        return 0.0
    delta = current - previous
    if delta <= 0:
        return 0.0
    return delta / elapsed
```

进程采样（名称缓存 + 只对展示条目取状态）：

```
for proc in psutil.process_iter(["pid", "cpu_percent", "memory_info"]):
    info = proc.info
    pid = info["pid"]
    if pid not in self._names:
        self._names[pid] = self._name_of(proc, pid)
    memory = info.get("memory_info")
    samples.append((
        float(info.get("cpu_percent") or 0.0) / self._cpu_count,
        int(getattr(memory, "rss", 0) or 0),
        pid,
    ))

samples.sort(reverse=True)
for cpu, rss, pid in samples[: self._limit]:
    entry = ProcessEntry(pid, self._names[pid], self._status_of(pid), cpu, rss)
```